

微网与外部电网电力调控策略的预测优化模型

摘要

针对光伏出力、负载和电价变化下的微网购电问题，本文以 10 分钟为基本时段，建立统一的能量平衡与储能状态模型，并依次研究确定性日计划、预测误差下的日前计划、预报更新后的滚动调整以及波动电价决策。储能容量为 12000 kWh，允许储电范围为 1200 至 10800 kWh，充放电功率不超过 5000 kW，充电和放电效率均取 90%。

问题 1 建立线性规划模型，规定日初与日末储电量均为 6000 kWh。求得日购电量 59,482.6990 kWh、购电费 35,126.95 元，比无储能基准节省 26.90%。独立模型的原始与对偶目标差小于十亿分之一元，验证了确定性模型的最优性。

问题 2 以此前日期数据滚动训练预测器，通过 1 月验证选择岭回归，采用三个历史残差情景优化日前购电，再以当前净负荷执行储能校正。2025 年 2 月至 12 月总费用为 14,506,106.15 元，紧急购电量为 224,803.88 kWh。问题 3 在 0、6、12、18 时接收光伏预报，以相邻有效计划的增减量计费；采用全部更新的总费用为 13,968,861.90 元，比只用 0 时预报降低 4.61%。该结果支持在所测试策略中采用三个额外预报时点。

问题 4 分别检验已知当天价格曲线和仅使用历史价格预测两种信息条件。前者对应问题 4-2 和 4-3 费用分别为 15,249,408.93 元和 14,608,808.73 元，后者分别为 15,319,637.30 元和 14,732,259.05 元。全年策略通过能量守恒、储能边界、功率、跨日连续和费用核算检查。除问题 1 外，所得费用为给定预测与控制策略的历史回放结果，不宣称全局最优。

关键词：微网；储能；线性规划；滚动预测；情景优化；购电策略

1 问题重述与建模范围

微网利用外网购电、光伏发电和储能共同满足小区负载。问题 1 在负载、电价已知及光伏单日预测给定时，要求当天供电不低于负载且日末储电量回归日初。问题 2 每天零点锁定购电计划，供能不足时按当时电价的 5 倍紧急购电，其余计划量仍须付款。问题 3 允许在新的光伏预报发布后调整尚未执行的计划，并承担增加或取消交易的费用。问题 4 进一步考虑电价实时波动。

第一问采用附件 1 和附录 1。第二问采用附件 1 电价及附件 2 实际序列；第三问增加附件 3；第四问以附件 4 替换价格序列。附件 2 至 4 覆盖 2025 年全年，模板要求正式输出 2 月 1 日至 12 月 31 日，共 334 天、48096 个时段。1 月用于预测选型和储能预运行，不能把后续实际数据用于早先决策。

1.1 时间和能量口径

附件每天有 144 个采样点，首个为 0:10，末个为次日 0:00。本文假设每个采样值代表此前 10 分钟区间的平均功率，采样时刻为右端点。因而 10:10 行对应 10:00—10:10。原结果模板末行或未列带有次日标记，现按内部实际计算含义统一标为本日 00:00—00:10；其他列仍保留原顺序。该处理是本文对时间歧义的解释，并非题面明示或官方勘误。

$$\Delta t = \frac{1}{6} \text{ h}, E_{d,t}^L = L_{d,t} \Delta t, E_{d,t}^{pv} = P_{d,t}^{pv} \Delta t \quad (1)$$

1.2 基本假设

充电量指母线送入储能的电量，放电量指储能向母线输出的电量，两侧效率分别为 0.9，往返效率为 0.81。题面若将 90% 理解为往返效率，可令两侧效率均为平方根 0.9，作为敏感性情形。忽略自放电与设备衰减；无售电收益、无外网购电功率上限。允许弃光和已付费富余电量弃置，以明确表达供电不得低于负载的约束。

第一问日初储电量采用附录给定的 6000 kWh。其余问题从 2025 年 1 月 1 日 6000 kWh 开始连续运行，日末状态传递给下一日，不额外强加每天首尾相等。实时策略仅使用当前及此前已观测的净负荷，未来观测只能用于事后评价。

2 符号与统一物理模型

符号	含义	单位
d, t	日期与日内时段索引，时段 0 至 143	—
$L_{d,t}, P_{d,t}^{pv}$	负载功率与光伏功率	kW
$p_{d,t}, \hat{p}_{d,t}$	实际价格与决策价格预测	元/kWh
$G_{d,t}^0, G_{d,t}$	日前计划量与最终有效常规购电量	kWh
$C_{d,t}, D_{d,t}$	母线侧充电量与放电量	kWh
$S_{d,t}$	时段开始时的储电量	kWh
$R_{d,t}, W_{d,t}$	紧急购电量与弃置电量	kWh
$\delta_{d,t}^{k,+}, \delta_{d,t}^{k,-}$	第 k 次调整的增购量与减购量	kWh
η_c, η_d	充电效率与放电效率，均为 0.9	—

令每个时段的供给加上紧急补购等于负载、充电和弃置之和。弃置变量使模型能表示光伏或已付费计划量的过剩，而不将其误算为负购电收入。

$$G_{d,t} + E_{d,t}^{pv} + D_{d,t} + R_{d,t} = E_{d,t}^L + C_{d,t} + W_{d,t} \quad (2)$$

$$S_{d,t+1} = S_{d,t} + \eta_c C_{d,t} - \frac{D_{d,t}}{\eta_d} \quad (3)$$

$$1200 \leq S_{d,t} \leq 10800, 0 \leq C_{d,t}, D_{d,t} \leq \frac{5000}{6} \quad (4)$$

$$G_{d,t}, R_{d,t}, W_{d,t} \geq 0, S_{d+1,0} = S_{d,144} \quad (5)$$

程序对每一个 10 分钟时段检查能量平衡与储能状态方程，再检查边界和日际衔接。六个 4 小时区间的充放电量是 24 个时段之和，不能仅凭 4 小时汇总推断区间内是否存在功率越限。因而验证使用逐时段轨迹，而不仅检查结果表。

3 数据处理与因果预测

3.1 数据完整性

附件 1 有 432 个建模数值；附件 2 负载、光伏和附件 4 价格各有 52560 个数值；附件 3 有 35040 个预报值，共 193152 个数值。数值区均无缺失、负值和非有限数。附件 3 的日期只在每组首行出现，组内空白通过向下继承日期解析，不能当作预测缺失。预报第 h 小时对应发布时间加 h 小时，跨日预测须按目标时间索引。

3.2 预测与选型

比较历史同时间均值、7 日、14 日、30 日移动平均、EWMA 及岭回归。第 d 天的预测只能读取前 d 天数据。岭回归使用前 1 日、前 7 日、近 7 日均值、周周期正余弦及截距，拟合窗口最多 60 天，功率统一除以 5000 缩放。非截距正则系数为 1，截距正则系数为百万分之一。训练不足 12 天时使用 7 日均值。预测值裁剪为非负，且上界只依据历史范围。

$$\hat{\beta} = (X^T X + \Lambda)^{-1} X^T y, \hat{y}_{d,t} = 5000 x_{d,t}^T \hat{\beta} \quad (6)$$

预测器按滚动起点验证原则构建[1]。经济选型实际采用 1 月 17 日至 31 日连续 15 天，候选为 6 种预测方法、3 种裕度和 3 种终端系数，共 54 组；以期末库存校正后的费用选出岭回归、裕度 0、终端系数 0.5。实际 1 月预运行仍使用事先固定的 EWMA、裕度 0.5 和终端系数 0.5，避免用月底选型反写月初状态。

1 月 1 日没有历史样本时，原实现以附件 1 的负载和光伏曲线冷启动。这是额外的初始化假设，并非附件 2 当日真实观测；正式评价从 2 月开始，当前复算保留此假设以与原结果一致。

方法	负载 MAE	负载 RMSE	光伏 MAE	光伏 RMSE
mean	803.65	1,028.24	483.33	772.97
ma7	781.21	955.92	153.40	303.86
ma14	782.67	959.83	171.48	327.22
ma30	789.76	973.51	225.65	400.08
ewma	838.19	1,039.77	154.73	303.83
ridge	180.32	242.31	150.46	302.77

表中误差单位均为 kW，评价区间为 2 月至 12 月。MAPE 仅作为辅助描述，排除接近零的实际功率；阈值采用全评价样本峰值的 1%，因此该指标不用于当时可用的在线特征。模型选择没有使用外样本误差反选参数。

3.3 三情景构造

最近 30 个历史整日的净负荷预测残差按全天能量排序，取排序位置约 15%、50%、85% 的三条完整残差曲线，以等权构成代表情景。它保留部分日内相关结构，但三个代表日不等同于经过校准的真实概率分布。

$$\hat{N}_{d,t} = (\hat{L}_{d,t} - \hat{P}_{d,t}^{pv}) \Delta t, N_{d,t}^{(s)} = \hat{N}_{d,t} + e_{d,t}^{(s)} + m \sigma_{d,t} \quad (7)$$

4 问题 1 的确定性优化模型

第一问在零点已获得当天负载、电价和光伏预测，取紧急购电为零。决策为 144 个常规购电量、144 个充电量、144 个放电量、144 个弃置量及 145 个储能状态。目标函数为真实购电费，附加首尾储能条件。

$$\min J_1 = \sum_{t=0}^{143} p_t G_t, R_t = 0, S_0 = S_{144} = 6000 \quad (8)$$

模型满足第 2 节各项约束，是线性规划，可由 HiGHS 求解[2,3]。原始实现另加一项微小吞吐惩罚以稳定选择解；本次独立求解去掉该项，并通过原始与对偶目标对照确认真实费用最优。两个模型的日购电量和费用一致。

$$J_\varepsilon = J_1 + 10^{-7} \sum_t (C_t + D_t) \quad (9)$$

在允许弃置且所有价格为正的模型中，若某时段同时充放电，可将充电减少 a 、放电减少 $0.81a$ ，从而保持储能变化不变。此时母线富余增加 $0.19a$ ，可通过减少购电或增加弃置保持平衡。真实购电费不会上升，吞吐惩罚严格减少。因此存在不同时充放电的最优解。若未来限制弃置或引入其他复杂费用，应重新验证这一推导，必要时加入二元互斥变量。

4.1 最优性与能量检验

指标	数值
日购电量 / kWh	59,482.6990
日购电费 / 元	35,126.948589
对偶目标 / 元	35,126.948589
原始对偶差 / 元	7.276e-12
无储能费用 / 元	48,052.05
费用节省比例	26.90%
日弃置电量 / kWh	0.0000

全天充电量为 20,740.6661 kWh，放电量为 16,799.9396 kWh。二者的差额为储能损耗，并非能量守恒错误。日初与日末均为 6000 kWh；储电轨迹覆盖 1200 至 10800 kWh，最大充电功率 5000 kW，最大放电功率 4,293.9182 kW。

$$\sum_t G_t + \sum_t E_t^{pv} - \sum_t E_t^L - \sum_t W_t = (1 - \eta_c) \sum_t C_t + (\eta_d^{-1} - 1) \sum_t D_t \quad (10)$$

该恒等式使用首尾储电量相等条件。独立模型的全日能量残差小于十亿分之一 kWh，双单纯形和内点算法所得费用一致。

4.2 问题 1 的指定结果与调度解释

表 1 问题 1 指定时段购电量和全天结果

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	480.4124	14:00-14:10	0.0000
16:00-16:10	445.4317	18:00-18:10	531.8940	20:00-20:10	0.0000
全天购电量	59,482.6990	全天购电费	35,126.95		

表 2 问题 1 储能充放电量和首尾储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	4,500.0000	0.0000	4:00-8:00	833.3333	6,365.8412
8:00-12:00	4,787.9643	1,702.9970	12:00-16:00	5,286.0352	91.1014
16:00-20:00	0.0000	5,780.1319	20:00-24:00	5,333.3333	2,859.8681
0:00 储电量	6,000.0000		24:00 储电量	6,000.0000	

表 1 电量单位为 kWh、费用单位为元，表 2 全部为 kWh。中午即使光伏高于负载，仍可能在较低价格时购电充电，故 12:00—12:10 购电不为零并非计算错误。晚高价时段放电能减少高价购电，但功率和容量约束使部分高价时段仍需从外网购电。

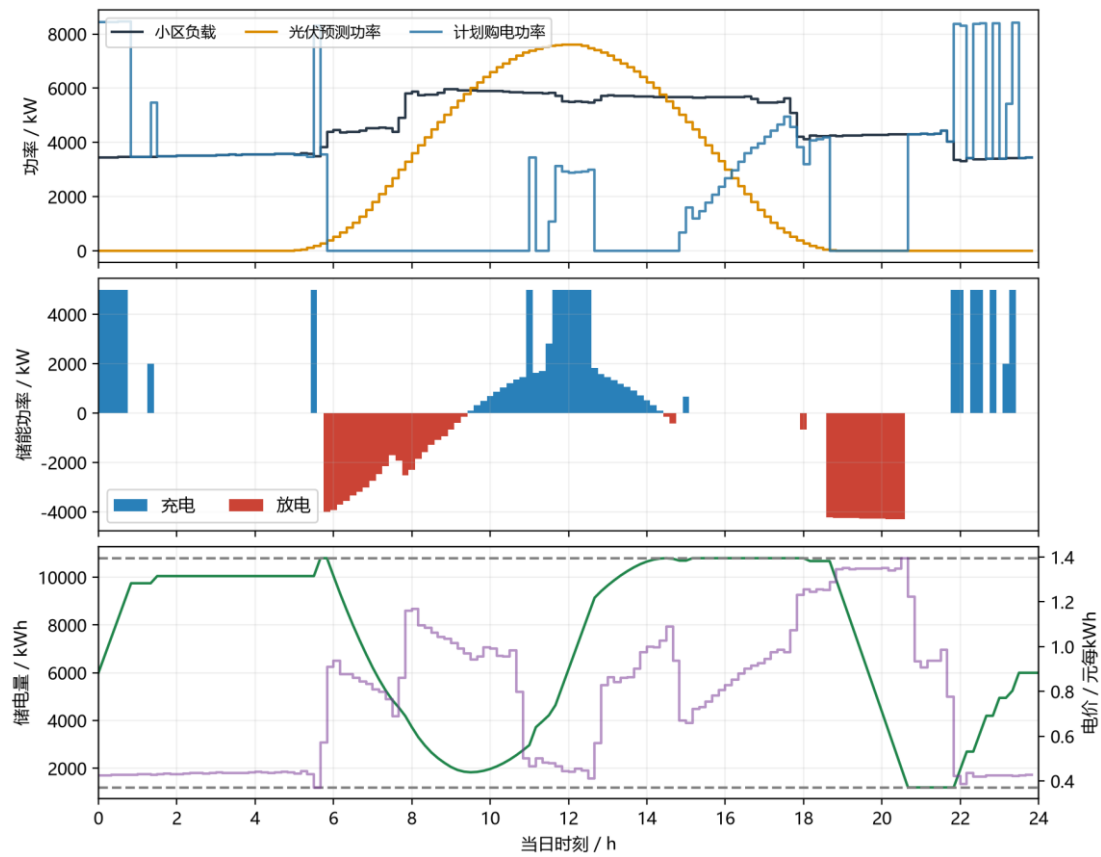


图 1 问题 1 购电与储能轨迹

5 问题 2 的日前计划与实时执行

未来负载和光伏未知时，不能直接把当天实际净负荷代入第一问模型。日前购电在各情景间共享，情景内储能、紧急购电、弃置和状态可不同。采用期望紧急费用与终端储能偏差惩罚构造以下近似规划。

$$\min \sum_t p_t G_t + \frac{1}{3} \sum_{s=1}^3 \left[\sum_t 5 p_t R_t^{(s)} + \kappa z_s \right] \quad (11)$$

$$z_s \geq S_{144}^{(s)} - 6000, z_s \geq 6000 - S_{144}^{(s)}, \kappa = 0.5\bar{p} \quad (12)$$

各情景均满足能量平衡和储能方程，另有极小吞吐惩罚。这里的情景内全天动作可以随整条情景变化，因此它是用于选择日前购电的两阶段近似，不是具有完整非预见性约束的多阶段最优控制。实际执行不使用任一情景的未来信息，也不照搬情景内电池轨迹。

5.1 实时可执行规则

在当前净负荷观测到达后，先用已锁定购电抵消净负荷。富余时尽量充电，缺口时在功率及储电下限内尽量放电，其余缺口紧急补购。下式中的储电量为当前时段起点状态。

$$x_t = G_t - N_t, C_t = \min\{x_t, 5000/6, (10800 - S_t)/\eta_c\} (x_t \geq 0) \quad (13)$$

$$D_t = \min\{-x_t, 5000/6, \eta_d(S_t - 1200)\}, R_t = -x_t - D_t (x_t < 0) \quad (14)$$

富余分支中放电和紧急量为零，未吸收富余全部弃置；缺口分支中充电和弃置为零。该规则是因果、可执行的贪心控制，但不能保证跨时段费用最优，因为提前放电可能减少后续高价时段的储能。

5.2 334 天汇总与结果口径

计划购电量 / kWh	计划购电费 / 元	紧急购电量 / kWh	紧急购电费 / 元
21,282,449.77	13,206,734.13	224,803.88	1,299,372.03

总费用为 14,506,106.15 元。2 月 1 日零点储电量为 10800 kWh，12 月 31 日末为 1,498.62 kWh。以下四页按题设表 1、表 2 列出指定日期结果，并列出了全部合并后的紧急购电区间。表 1 的全天购电量指计划常规购电，紧急购电单独列示；费用同时给出计划费和含紧急购电的总费。

5.3 2025-03-20 的指定结果

电量单位为 kWh，费用单位为元。日前计划在 0:00 确定，储能统计为实际执行结果。相邻紧急购电时段合并为连续区间，所列电量为区间累计。

表 1 格式 日前计划购电量

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	771.3945	14:00-14:10	0.0000
16:00-16:10	461.5890	18:00-18:10	695.3051	20:00-20:10	0.0000
全天计划量	66,797.9104	全天计划费	39,990.10	全天总费用	47,364.95

表 2 格式 实际储能充放电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	8,550.4921	406.8213	04:00-08:00	1,996.3357	5,894.0913
08:00-12:00	6,268.6252	2,520.6772	12:00-16:00	4,640.2298	684.5976
16:00-20:00	167.7293	6,084.6602	20:00-24:00	365.9685	2,353.2793
0:00 储电量	1,509.6215		24:00 储电量	1,362.1453	

紧急购电明细

时间段	紧急购电量
09:10-10:00	256.7000
20:20-21:50	952.9489

全天紧急购电 1,209.6489 kWh，紧急购电费 7,374.85 元；常规购电费加紧急购电费等于总费用 47,364.95 元。4 小时内出现充、放电累计值均为正，表示不同 10 分钟时段切换运行，不表示同时充放电。

5.3 2025-06-21 的指定结果

电量单位为 kWh，费用单位为元。日前计划在 0:00 确定，储能统计为实际执行结果。相邻紧急购电时段合并为连续区间，所列电量为区间累计。

表 1 格式 日前计划购电量

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	0.0000	14:00-14:10	0.0000
16:00-16:10	243.6957	18:00-18:10	429.7072	20:00-20:10	0.0000
全天计划量	38,371.2373	全天计划费	23,723.39	全天总费用	23,723.39

表 2 格式 实际储能充放电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	1,773.2305	45.9992	04:00-08:00	722.6568	2,622.4424
08:00-12:00	6,801.7663	0.0000	12:00-16:00	0.0000	0.0000
16:00-20:00	17.6848	4,209.7206	20:00-24:00	2,621.9394	2,418.8181
0:00 储电量	5,397.0469		24:00 储电量	5,810.6188	

紧急购电明细

时间段	紧急购电量
无	0.0000

全天紧急购电 0.0000 kWh，紧急购电费 0.00 元；常规购电费加紧急购电费等于总费用 23,723.39 元。4 小时内出现充、放电累计值均为正，表示不同 10 分钟时段切换运行，不表示同时充放电。

5.3 2025-09-23 的指定结果

电量单位为 kWh，费用单位为元。日前计划在 0:00 确定，储能统计为实际执行结果。相邻紧急购电时段合并为连续区间，所列电量为区间累计。

表 1 格式 日前计划购电量

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	458.3946	14:00-14:10	0.0000
16:00-16:10	514.8372	18:00-18:10	639.0266	20:00-20:10	0.0000
全天计划量	69,189.7195	全天计划费	43,401.10	全天总费用	45,264.15

表 2 格式 实际储能充放电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	9,496.4946	18.1704	04:00-08:00	1,536.4526	5,269.0828
08:00-12:00	4,752.3260	1,896.6157	12:00-16:00	4,139.6910	575.6187
16:00-20:00	136.1602	6,010.1989	20:00-24:00	668.0329	2,676.1702
0:00 储电量	1,334.9402		24:00 储电量	1,718.0078	

紧急购电明细

时间段	紧急购电量
21:00-21:50	407.3285

全天紧急购电 407.3285 kWh，紧急购电费 1,863.05 元；常规购电费加紧急购电费等于总费用 45,264.15 元。4 小时内出现充、放电累计值均为正，表示不同 10 分钟时段切换运行，不表示同时充放电。

5.3 2025-12-21 的指定结果

电量单位为 kWh，费用单位为元。日前计划在 0:00 确定，储能统计为实际执行结果。相邻紧急购电时段合并为连续区间，所列电量为区间累计。

表 1 格式 日前计划购电量

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	1,110.7036	14:00-14:10	0.0000
16:00-16:10	794.0558	18:00-18:10	684.0892	20:00-20:10	0.0000
全天计划量	100,381.7467	全天计划费	64,356.26	全天总费用	65,394.16

表 2 格式 实际储能充放电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	9,004.0027	70.9169	04:00-08:00	1,799.9906	1,461.9493
08:00-12:00	5,667.3219	7,064.8587	12:00-16:00	6,743.7069	2,203.0187
16:00-20:00	85.1565	6,001.5592	20:00-24:00	613.3872	2,737.6507
0:00 储电量	1,892.1132		24:00 储电量	1,703.2630	

紧急购电明细

时间段	紧急购电量
20:30-20:40	148.7820

全天紧急购电 148.7820 kWh，紧急购电费 1,037.90 元；常规购电费加紧急购电费等于总费用 65,394.16 元。4 小时内出现充、放电累计值均为正，表示不同 10 分钟时段切换运行，不表示同时充放电。

6 问题 3 的预报更新与购电调整

在 0、6、12、18 时收到的 24 个预报值依次对应发布后第 1 至第 24 个整点。用发布时间已知的光伏实测值作为左端锚点，逐段线性插值生成 10 分钟预测；只取当日尚未执行部分。历史误差使用同一发布时间的历史预报残差，避免把日前误差直接套用为临近预报误差。

负载没有同步发布的外部预报，故将最近 1 小时已经实现的负载残差均值加到日前预测，并按 6 小时尺度指数衰减。该衰减尺度是建模设定，未进行独立最优性证明。

$$\hat{L}_t^r = \max\{0, \hat{L}_t^0 + \bar{e}_r \exp[-(t - 6\tau)/36]\} \quad (15)$$

每次更新只重算剩余时段，已执行购电和储能状态冻结。程序实施的是 6 小时预报触发的购电滚动优化，电池仍每 10 分钟按第 5 节贪心规则执行。因而不能称为每 10 分钟都重新优化电池的完整 MPC；这里借鉴的是滚动时域思想[4]。

$$\delta_t^{k,+} = \max\{G_t^k - G_t^{k-1}, 0\}, \delta_t^{k,-} = \max\{G_t^{k-1} - G_t^k, 0\} \quad (16)$$

$$J_3 = \sum_t p_t G_t^0 + \sum_{k,t} (1.5 p_t \delta_t^{k,+} - 0.5 p_t \delta_t^{k,-}) + \sum_t 5 p_t R_t \quad (17)$$

题面未明确取消部分是否退回原款，也未明确多次调整以最初计划还是上一有效计划为基准。主模型明确约定：取消部分退回原购电款，再支付 50% 违约金；每次按上一有效计划结算。另以不退款情形作敏感性。增购费减去取消净退款可为净调整费，不能把它等同于纯粹违约附加费。

$$G_t^{\text{final}} = G_t^0 + \sum_k \delta_t^{k,+} - \sum_k \delta_t^{k,-} \quad (18)$$

$$J_3 = \sum_t p_t G_t^{\text{final}} + 0.5 \sum_{k,t} p_t (\delta_t^{k,+} + \delta_t^{k,-}) + \sum_t 5 p_t R_t \quad (19)$$

6.1 更新策略的经济比较

策略	发布时刻	总费用 / 元	紧急量 / kWh	净调整费 / 元
A	0	14,643,207.45	220,412.16	0.00
B	0、6	14,265,453.54	94,010.96	389,089.89
C	0、6、12	14,083,138.95	66,497.09	383,298.59
D	0、6、12、18	13,968,861.90	28,257.01	453,886.70

按照 A→B→C→D 的嵌套比较，增加 6 时、12 时、18 时更新分别降低费用 377753.91 元、182314.59 元、114277.05 元。三次增量在本回放中均为正，因此在题面给出的四个时点中，保留全部更新有经济依据。该比较不是各时点彼此独立的价值分解，也没有评估题面以外新时点的实际预报。

策略 D 相对 A 降低总费用 4.61%，紧急购电量降低 87.18%。由于策略之间日末储能不同，比较是实际账单口径；终端库存价值差相对总额较小，但仍须在解释中说明。敏感性实验另以统一的库存价值进行校正。

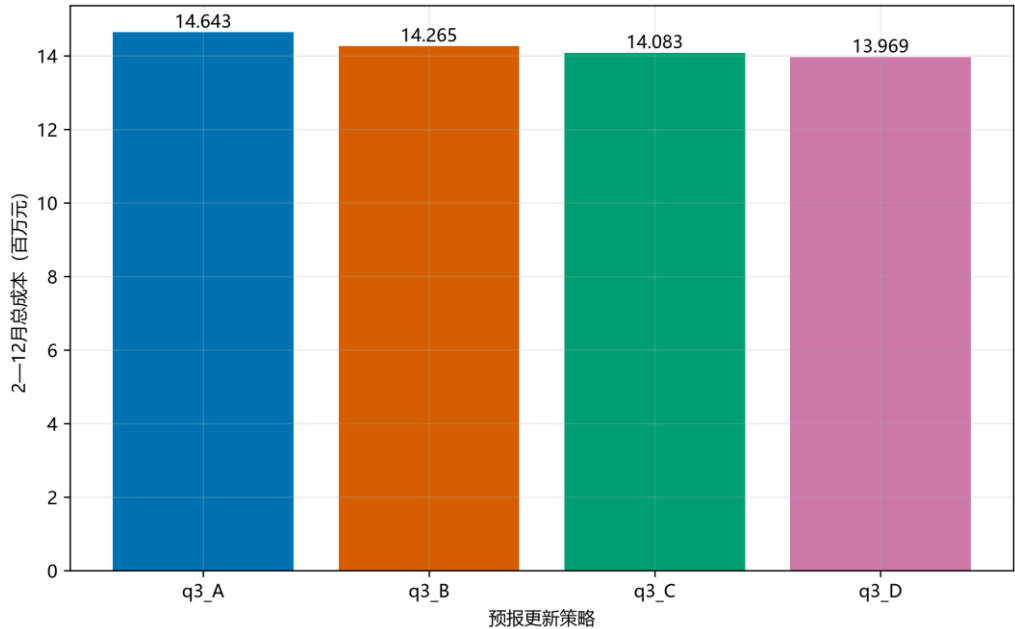


图 2 不同更新策略的累计费用

6.2 信息价值的适用边界

预报更准确并不自动保证账单更低，调整费、已执行状态和实时贪心控制都可能影响结果。本数据中额外更新有利，只能支持这一组策略和费用约定；不能推出任意更频繁更新都更好。若拟引入其他发布时刻，需先获得这些时刻真实可用的预报，再做相同信息边界和费用口径下的留出比较。

以下四页同时给出 0 时计划与调整后的最终常规量，避免将“调整购电量”误解为仅记录增量。净调整费已经包括增购、违约和退款，紧急费另外计入总费用。

6.3 2025-03-20 的指定结果

电量单位为 kWh，费用单位为元。日前计划在 0:00 确定，储能统计为实际执行结果。相邻紧急购电时段合并为连续区间，所列电量为区间累计。

表 1 格式 日前计划购电量

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	667.6990	14:00-14:10	0.0000
16:00-16:10	614.4013	18:00-18:10	683.5314	20:00-20:10	0.0000
全天计划量	68,546.7593	全天计划费	41,927.74	全天总费用	46,057.96

补充表 调整后的最终常规购电量

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	530.0919	14:00-14:10	0.0000
16:00-16:10	524.5994	18:00-18:10	657.3709	20:00-20:10	0.0000
全天最终量	66,290.7361	净调整费	3,100.82	最终常规费	45,028.57

表 2 格式 实际储能充放电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	7,901.5416	312.4061	04:00-08:00	1,936.1898	5,992.9425
08:00-12:00	4,286.4608	2,565.1168	12:00-16:00	6,656.1390	294.4557
16:00-20:00	174.5376	5,709.0510	20:00-24:00	586.5721	3,044.0926
0:00 储电量	2,202.1144		24:00 储电量	1,680.4504	

紧急购电明细

时间段	紧急购电量
09:10-10:00	212.2604

全天紧急购电 212.2604 kWh，紧急购电费 1,029.39 元；常规购电费加紧急购电费等于总费用 46,057.96 元。4 小时内出现充、放电累计值均为正，表示不同 10 分钟时段切换运行，不表示同时充放电。

6.3 2025-06-21 的指定结果

电量单位为 kWh，费用单位为元。日前计划在 0:00 确定，储能统计为实际执行结果。相邻紧急购电时段合并为连续区间，所列电量为区间累计。

表 1 格式 日前计划购电量

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	0.0000	14:00-14:10	0.0000
16:00-16:10	124.7031	18:00-18:10	421.8987	20:00-20:10	0.0000
全天计划量	39,195.5367	全天计划费	23,534.86	全天总费用	23,085.25

补充表 调整后的最终常规购电量

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	0.0000	14:00-14:10	0.0000
16:00-16:10	142.4231	18:00-18:10	0.0000	20:00-20:10	0.0000
全天最终量	37,757.0008	净调整费	-449.60	最终常规费	23,085.25

表 2 格式 实际储能充放电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	2,025.1162	0.0000	04:00-08:00	1,380.7565	1,799.5612
08:00-12:00	7,883.0107	0.0000	12:00-16:00	0.0000	0.0000
16:00-20:00	3.3184	5,533.8769	20:00-24:00	2,357.3932	2,506.6821
0:00 储电量	2,639.5173		24:00 储电量	3,990.6860	

紧急购电明细

时间段	紧急购电量
无	0.0000

全天紧急购电 0.0000 kWh，紧急购电费 0.00 元；常规购电费加紧急购电费等于总费用 23,085.25 元。4 小时内出现充、放电累计值均为正，表示不同 10 分钟时段切换运行，不表示同时充放电。

6.3 2025-09-23 的指定结果

电量单位为 kWh，费用单位为元。日前计划在 0:00 确定，储能统计为实际执行结果。相邻紧急购电时段合并为连续区间，所列电量为区间累计。

表 1 格式 日前计划购电量

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	439.0507	14:00-14:10	0.0000
16:00-16:10	582.8025	18:00-18:10	676.0235	20:00-20:10	0.0000
全天计划量	72,730.8991	全天计划费	45,783.96	全天总费用	44,979.24

补充表 调整后的最终常规购电量

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	320.2309	14:00-14:10	0.0000
16:00-16:10	520.8383	18:00-18:10	676.0235	20:00-20:10	3.6316
全天最终量	67,830.1518	净调整费	-1,483.70	最终常规费	44,300.26

表 2 格式 实际储能充放电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	8,522.5849	76.4939	04:00-08:00	1,864.4549	6,721.1732
08:00-12:00	5,898.2149	1,896.6157	12:00-16:00	4,937.7984	412.1953
16:00-20:00	182.4415	5,975.3091	20:00-24:00	925.2721	2,805.0885
0:00 储电量	1,808.0385		24:00 储电量	2,031.4222	

紧急购电明细

时间段	紧急购电量
20:20-20:30	6.3750
21:20-21:50	138.4320

全天紧急购电 144.8070 kWh，紧急购电费 678.98 元；常规购电费加紧急购电费等于总费用 44,979.24 元。4 小时内出现充、放电累计值均为正，表示不同 10 分钟时段切换运行，不表示同时充放电。

6.3 2025-12-21 的指定结果

电量单位为 kWh，费用单位为元。日前计划在 0:00 确定，储能统计为实际执行结果。相邻紧急购电时段合并为连续区间，所列电量为区间累计。

表 1 格式 日前计划购电量

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	952.2926	14:00-14:10	0.0000
16:00-16:10	774.1051	18:00-18:10	730.2102	20:00-20:10	0.0000
全天计划量	97,742.2025	全天计划费	62,649.23	全天总费用	64,500.87

补充表 调整后的最终常规购电量

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	952.7962	14:00-14:10	0.0000
16:00-16:10	852.7269	18:00-18:10	730.2102	20:00-20:10	0.0000
全天最终量	97,256.8432	净调整费	1,781.88	最终常规费	64,431.10

表 2 格式 实际储能充放电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	9,014.6331	42.3126	04:00-08:00	1,882.7950	1,669.3056
08:00-12:00	5,494.4980	7,236.4164	12:00-16:00	7,333.5369	2,306.5261
16:00-20:00	20.6683	5,541.6255	20:00-24:00	444.7648	2,994.5754
0:00 储电量	1,952.1507		24:00 储电量	1,734.2221	

紧急购电明细

时间段	紧急购电量
07:50-08:00	12.0105

全天紧急购电 12.0105 kWh，紧急购电费 69.76 元；常规购电费加紧急购电费等于总费用 64,500.87 元。4 小时内出现充、放电累计值均为正，表示不同 10 分钟时段切换运行，不表示同时充放电。

7 问题 4 的波动电价模型

附件 4 提供实际发生的电价，并没有说明整条当日价格在 0 时已经公开。因此本文并列报告两个信息条件：已知当天价格的条件基准，以及以历史价格预测进行决策的可实施扩展。保留原五份结果表对应的条件基准，避免把假设变化混同于数值纠错。

$$\hat{p}_{d,t} = \frac{1}{\min(7,d)} \sum_{j=\max(0,d-7)}^{d-1} p_{j,t} \quad (d \geq 1) \quad (20)$$

历史预测扩展仅用此前 7 天同时间价格，第一天用附件 1 电价作冷启动。决策时把模型中的价格换成预测值，实际结算始终使用附件 4 实际价格。由于没有附加的日内价格预报，在 6、12、18 时仍使用零点生成的价格预测，不读取未来实际价格。

策略	价格信息	总费用 / 元	紧急量 / kWh
4-2	当天曲线已知	15,249,408.93	237,748.18
4-3	当天曲线已知	14,608,808.73	36,078.78
4-2 扩展	历史 7 日均价	15,319,637.30	230,283.12
4-3 扩展	历史 7 日均价	14,732,259.05	34,167.83

历史价格预测使问题 4-2 和 4-3 费用分别比对应条件基准高 0.46% 和 0.85%。这是两种价格信息输入下同类启发式策略的实测费用差，并非严格的信息价值上界或全局最优差距。要获得信息价值界，需要在一致终端条件下建立可比较的最优随机控制与完美信息模型。

已知当天曲线的 4-3 费用比 4-2 低 4.20%，历史价格预测扩展的对应降幅为 3.83%。额外光伏预报的应用在这两组测试中仍能减少费用，但该降幅同时反映预测器与滚动执行的综合效果。下面八页保留与原结果工作簿一致的已知价格条件下指定日期结果。

7.1 问题 4-2 2025-03-20 的指定结果

电量单位为 kWh，费用单位为元。日前计划在 0:00 确定，储能统计为实际执行结果。相邻紧急购电时段合并为连续区间，所列电量为区间累计。

表 1 格式 日前计划购电量

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	771.3945	14:00-14:10	0.0000
16:00-16:10	461.5890	18:00-18:10	0.0000	20:00-20:10	0.0000
全天计划量	67,716.3431	全天计划费	41,113.08	全天总费用	48,252.47

表 2 格式 实际储能充放电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	6,706.3072	512.6354	04:00-08:00	3,753.2098	5,752.8951
08:00-12:00	6,268.6252	2,548.0120	12:00-16:00	4,632.0511	719.4601
16:00-20:00	144.4381	6,685.6107	20:00-24:00	1,868.8262	2,214.3473
0:00 储电量	1,579.2597		24:00 储电量	2,134.3043	

紧急购电明细

时间段	紧急购电量
09:10-10:00	229.3652
20:20-21:50	965.9920

全天紧急购电 1,195.3572 kWh，紧急购电费 7,139.39 元；常规购电费加紧急购电费等于总费用 48,252.47 元。4 小时内出现充、放电累计值均为正，表示不同 10 分钟时段切换运行，不表示同时充放电。

7.1 问题 4-2 2025-06-21 的指定结果

电量单位为 kWh，费用单位为元。日前计划在 0:00 确定，储能统计为实际执行结果。相邻紧急购电时段合并为连续区间，所列电量为区间累计。

表 1 格式 日前计划购电量

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	0.0000	14:00-14:10	0.0000
16:00-16:10	243.6957	18:00-18:10	429.7072	20:00-20:10	0.0000
全天计划量	39,414.3625	全天计划费	20,287.91	全天总费用	20,287.91

表 2 格式 实际储能充放电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	1,982.4727	3,695.4480	04:00-08:00	3,305.5459	1,719.9540
08:00-12:00	7,621.6876	0.0000	12:00-16:00	0.0000	0.0000
16:00-20:00	15.6893	4,170.5899	20:00-24:00	3,258.6098	3,117.4407
0:00 储电量	5,198.3777		24:00 储电量	5,649.0575	

紧急购电明细

时间段	紧急购电量
无	0.0000

全天紧急购电 0.0000 kWh，紧急购电费 0.00 元；常规购电费加紧急购电费等于总费用 20,287.91 元。4 小时内出现充、放电累计值均为正，表示不同 10 分钟时段切换运行，不表示同时充放电。

7.1 问题 4-2 2025-09-23 的指定结果

电量单位为 kWh，费用单位为元。日前计划在 0:00 确定，储能统计为实际执行结果。相邻紧急购电时段合并为连续区间，所列电量为区间累计。

表 1 格式 日前计划购电量

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	458.3946	14:00-14:10	0.0000
16:00-16:10	514.8372	18:00-18:10	0.0000	20:00-20:10	739.2926
全天计划量	69,181.2047	全天计划费	44,589.89	全天总费用	46,811.69

表 2 格式 实际储能充放电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	6,510.9961	18.2573	04:00-08:00	4,481.1549	5,232.7774
08:00-12:00	4,825.7005	1,896.6157	12:00-16:00	4,184.4661	693.7682
16:00-20:00	254.9077	6,868.2389	20:00-24:00	791.2947	2,027.5618
0:00 储电量	1,331.4140		24:00 储电量	1,678.1716	

紧急购电明细

时间段	紧急购电量
20:30-20:40	9.4001
20:50-21:50	442.9388

全天紧急购电 452.3389 kWh，紧急购电费 2,221.80 元；常规购电费加紧急购电费等于总费用 46,811.69 元。4 小时内出现充、放电累计值均为正，表示不同 10 分钟时段切换运行，不表示同时充放电。

7.1 问题 4-2 2025-12-21 的指定结果

电量单位为 kWh，费用单位为元。日前计划在 0:00 确定，储能统计为实际执行结果。相邻紧急购电时段合并为连续区间，所列电量为区间累计。

表 1 格式 日前计划购电量

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	1,110.7036	14:00-14:10	0.0000
16:00-16:10	794.0558	18:00-18:10	684.0892	20:00-20:10	0.0000
全天计划量	99,075.3136	全天计划费	75,314.29	全天总费用	75,314.29

表 2 格式 实际储能充放电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	5,904.9303	70.9169	04:00-08:00	806.9479	1,452.3576
08:00-12:00	5,808.2857	7,205.8225	12:00-16:00	7,086.1338	2,487.4074
16:00-20:00	126.8923	5,314.2748	20:00-24:00	3,704.2350	2,249.9071
0:00 储电量	5,564.3592		24:00 储电量	5,790.6127	

紧急购电明细

时间段	紧急购电量
无	0.0000

全天紧急购电 0.0000 kWh，紧急购电费 0.00 元；常规购电费加紧急购电费等于总费用 75,314.29 元。4 小时内出现充、放电累计值均为正，表示不同 10 分钟时段切换运行，不表示同时充放电。

7.2 问题 4-3 2025-03-20 的指定结果

电量单位为 kWh，费用单位为元。日前计划在 0:00 确定，储能统计为实际执行结果。相邻紧急购电时段合并为连续区间，所列电量为区间累计。

表 1 格式 日前计划购电量

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	667.6990	14:00-14:10	0.0000
16:00-16:10	614.4013	18:00-18:10	0.0000	20:00-20:10	0.0000
全天计划量	69,440.3509	全天计划费	43,015.62	全天总费用	47,287.13

补充表 调整后的最终常规购电量

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	530.0919	14:00-14:10	0.0000
16:00-16:10	524.5994	18:00-18:10	0.0000	20:00-20:10	0.0000
全天最终量	67,169.0154	净调整费	3,247.08	最终常规费	46,262.70

表 2 格式 实际储能充放电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	6,239.7904	504.1176	04:00-08:00	3,430.3878	5,757.7905
08:00-12:00	4,283.9900	2,575.5103	12:00-16:00	6,658.6098	294.4557
16:00-20:00	174.5376	6,377.8626	20:00-24:00	1,992.3802	2,899.4282
0:00 储电量	2,316.1934		24:00 储电量	2,363.2918	

紧急购电明细

时间段	紧急购电量
09:10-10:00	201.8669

全天紧急购电 201.8669 kWh，紧急购电费 1,024.43 元；常规购电费加紧急购电费等于总费用 47,287.13 元。4 小时内出现充、放电累计值均为正，表示不同 10 分钟时段切换运行，不表示同时充放电。

7.2 问题 4-3 2025-06-21 的指定结果

电量单位为 kWh，费用单位为元。日前计划在 0:00 确定，储能统计为实际执行结果。相邻紧急购电时段合并为连续区间，所列电量为区间累计。

表 1 格式 日前计划购电量

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	0.0000	14:00-14:10	0.0000
16:00-16:10	128.0093	18:00-18:10	438.5881	20:00-20:10	0.0000
全天计划量	39,781.5323	全天计划费	20,452.48	全天总费用	19,895.89

补充表 调整后的最终常规购电量

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	0.0000	14:00-14:10	0.0000
16:00-16:10	142.4231	18:00-18:10	406.9691	20:00-20:10	0.0000
全天最终量	38,346.9171	净调整费	-556.59	最终常规费	19,895.89

表 2 格式 实际储能充放电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	2,368.6485	1,687.5172	04:00-08:00	3,079.7811	1,778.2734
08:00-12:00	7,943.5653	0.0000	12:00-16:00	0.0000	0.0000
16:00-20:00	2.7747	5,537.9433	20:00-24:00	3,094.9299	3,177.7014
0:00 储电量	2,598.0832		24:00 储电量	3,903.8845	

紧急购电明细

时间段	紧急购电量
无	0.0000

全天紧急购电 0.0000 kWh，紧急购电费 0.00 元；常规购电费加紧急购电费等于总费用 19,895.89 元。4 小时内出现充、放电累计值均为正，表示不同 10 分钟时段切换运行，不表示同时充放电。

7.2 问题 4-3 2025-09-23 的指定结果

电量单位为 kWh，费用单位为元。日前计划在 0:00 确定，储能统计为实际执行结果。相邻紧急购电时段合并为连续区间，所列电量为区间累计。

表 1 格式 日前计划购电量

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	439.0507	14:00-14:10	0.0000
16:00-16:10	582.8025	18:00-18:10	0.0000	20:00-20:10	805.4434
全天计划量	72,731.6962	全天计划费	47,105.84	全天总费用	46,311.76

补充表 调整后的最终常规购电量

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	324.7174	14:00-14:10	0.0000
16:00-16:10	520.8383	18:00-18:10	0.0000	20:00-20:10	756.5694
全天最终量	67,663.5085	净调整费	-1,569.54	最终常规费	45,536.30

表 2 格式 实际储能充放电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	6,212.4251	165.5270	04:00-08:00	4,254.0343	6,680.0303
08:00-12:00	5,403.7451	1,896.6157	12:00-16:00	5,426.0405	412.1953
16:00-20:00	269.1458	6,888.5569	20:00-24:00	935.5478	1,979.8807
0:00 储电量	1,795.3772		24:00 储电量	2,020.8820	

紧急购电明细

时间段	紧急购电量
18:20-18:30	1.0833
20:20-20:30	6.3750
21:20-21:50	146.6475

全天紧急购电 154.1058 kWh，紧急购电费 775.46 元；常规购电费加紧急购电费等于总费用 46,311.76 元。4 小时内出现充、放电累计值均为正，表示不同 10 分钟时段切换运行，不表示同时充放电。

7.2 问题 4-3 2025-12-21 的指定结果

电量单位为 kWh，费用单位为元。日前计划在 0:00 确定，储能统计为实际执行结果。相邻紧急购电时段合并为连续区间，所列电量为区间累计。

表 1 格式 日前计划购电量

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	952.2926	14:00-14:10	0.0000
16:00-16:10	774.1051	18:00-18:10	730.2102	20:00-20:10	0.0000
全天计划量	97,555.7323	全天计划费	73,293.83	全天总费用	74,945.16

补充表 调整后的最终常规购电量

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	952.7962	14:00-14:10	0.0000
16:00-16:10	852.7269	18:00-18:10	683.5154	20:00-20:10	0.0000
全天最终量	96,887.3410	净调整费	1,566.88	最终常规费	74,860.71

表 2 格式 实际储能充放电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	5,763.9579	74.6913	04:00-08:00	779.2716	1,478.9757
08:00-12:00	5,467.7745	7,305.0077	12:00-16:00	7,651.3700	2,473.7335
16:00-20:00	0.0000	5,388.0558	20:00-24:00	4,353.3056	3,058.6478
0:00 储电量	5,695.4283		24:00 储电量	5,332.7488	

紧急购电明细

时间段	紧急购电量
07:50-08:00	12.0105

全天紧急购电 12.0105 kWh，紧急购电费 84.45 元；常规购电费加紧急购电费等于总费用 74,945.16 元。4 小时内出现充、放电累计值均为正，表示不同 10 分钟时段切换运行，不表示同时充放电。

8 验证与敏感性分析

12 组全年策略分别检查 11 项物理和费用指标，共 132 项。最大绝对误差为 $5.821\text{e-}11$ ，小于百万分之一的验证阈值。检查包括能量平衡、储能动态、容量边界、功率、非负、充放电互斥、跨日连续、费用、交易恒等式、紧急区间合并与有限数值。

五份原始 Excel 已独立读取，与重建轨迹逐值比较；共比较 288720 个时段购电值，并同时检查储能汇总、日期、紧急区间及费用，最大差异为约 1.2 乘 10 的负 10 次方。61 项污染测试覆盖六种预测器与不同日期，并检查修改 12 时后实际值和后续预报不会影响此前计划。测试通过说明这些输入路径满足已测试的因果边界，不替代对所有控制逻辑的数学证明。

敏感性使用 3 月 18—23 日、6 月 19—24 日、9 月 21—26 日和 12 月 19—24 日共 24 天；组内保持相同初始储电，并扣除终端相对初始的库存价值。以下为问题 3-D 部分实验，不将 24 天结论外推为全年保证。

$$J_{\text{adj}} = J - 0.5\bar{p}(S_{\text{end}} - S_{\text{start}}) \quad (21)$$

情形	库存校正成本 / 元	相对基准
基准	1,022,316.93	+0.00%
效率各 85%	1,061,720.06	+3.85%
效率各 95%	987,555.12	-3.40%
往返效率 90%	988,531.27	-3.30%
容量 9600 kWh	1,072,606.20	+4.92%
容量 14400 kWh	978,885.00	-4.25%
残差裕度 1	1,093,889.71	+7.00%
取消不退款	1,030,790.03	+0.83%
PCHIP 插值	1,022,705.50	+0.04%

效率提高与容量扩大在所测范围内降低费用；过大安全裕度会增加常规购电和富余弃置。取消不退款使策略 3-D 的 24 天费用上升，但本实验没有在不退款条件下重新比较完整 A—D 组合，因此不能据此断言该条件下所有更新时点仍然有利。

9 模型评价与结论

统一能量模型能清楚区分功率和电量、母线流量和储能库存。第一问可获得线性规划最优解及对偶证据；全年策略按信息发布时间运行，提供可核算的计划、调整与紧急账本。输出结果覆盖题目指定日期、六个购电时段、六个 4 小时充放电区间及全天费用。

模型也有明确局限。三条历史残差情景不能完全描述极端天气风险，情景内动作未施加完整多阶段非预见性；实际电池执行为贪心规则，未优化未来的机会成本。无衰减、自放电和售电机假设使模型只适用于题设简化系统。问题 4 已知价格基准依赖额外信息条件，不能作为实时未知价格下的无条件答案。

可以在已有框架上把储能执行改为每 10 分钟基于条件预测的短时域优化，加入场景树的非预见性约束，并通过外样本净费用比较是否值得增加计算复杂度。若新增价格发布机制或改变退款条款，应重新求解，不能只改报表费率。

综合计算，问题 1 最优日购电费为 35,126.95 元；问题 2 和问题 3-D 在 2 月至 12 月的实际账单分别为 14,506,106.15 元和 13,968,861.90 元。增加题面提供的 6、12、18 时预报在所测嵌套策略中均节省费用。问题 4 两种价格信息条件下的费用已分别报告，保证结论与可用信息相匹配。

参考文献

- [1] Hyndman R J, Athanasopoulos G. Forecasting: Principles and Practice. 3rd ed. OTexts, 2021. <https://otexts.com/fpp3/>（访问日期 2026-09-10）。
- [2] SciPy Developers. `scipy.optimize.linprog`. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.linprog.html>（访问日期 2026-09-10）。
- [3] Boyd S, Vandenberghe L. Convex Optimization. Cambridge University Press, 2004. <https://web.stanford.edu/~boyd/cvxbook/>.
- [4] Rawlings J B, Mayne D Q, Diehl M M. Model Predictive Control: Theory, Computation, and Design. 2nd ed., 6th printing. Nob Hill Publishing, 2026. <https://sites.engineering.ucsb.edu/~jbrow/mpc/>.
- [5] 全国大学生数学建模竞赛组委会. 全国大学生数学建模竞赛论文格式规范（2026 年修订稿）. 2026-03-03. https://www.mcm.edu.cn/html_cn/node/4cd596519c9eb9fbd866398f6df0caa3.html.

附录 A 支撑材料与复现说明

计算源程序、输入附件、逐时段轨迹、费用明细、误差检验和公式源文件随支撑材料提供。原始结果和初稿保留，修订工作簿只修正与内部映射不一致的时间文字并增加费用核算页，已验证的购电数值保持不变。

文件或目录	内容
src/model.py	数据读取、预测、线性规划、实时执行与验证
src/run.py	问题 1、54 组经济选型、12 组年度策略及敏感性
src/q1_certificate.py	独立第一问模型、对偶证书和算法交叉检查
src/review_results.py	原始五份工作簿逐值独立回读
src/deliver.py 中 causality	61 项未来污染验证，原导出器保留供对照
results/*.npz 与 *_daily.csv	逐时段轨迹和逐日费用明细
audit/*.csv 与 *.json	数据审计、时间映射、物理与费用检验
论文公式 LaTeX.tex	本文公式的可编辑 LaTeX 源
src/word_math.py	LaTeX 至 Word 原生公式转换
src/build_review_paper.py	由复算结果生成本文

运行环境采用 Python 3.12、NumPy 2.5.3、SciPy 1.18.1、pandas 3.0.1；先执行 run.py 的 all 阶段，再执行 review_results.py 与因果测试。独立第一问验证使用 q1_certificate.py。所有路径相对于本复现目录解析，输入放在 input，计算结果写入 results 和 audit。

AI 工具使用说明

本次使用 OpenAI Codex 辅助代码审查、结果复算、论文文字修订、图表制作和公式转换。论文中的数值来自提供附件及可复现程序；数学假设和策略局限在正文注明。正式参赛时，使用者应依据实际交互记录完成赛事要求的 AI 工具使用记录，不应将本段当作完整的提示词与交互日志。

以下附上产生主要数值结果的完整模型与运行程序，以及独立回读程序和第一问最优性验证程序。排版与图表生成脚本一并列入支撑材料。

附录 B model.py 源程序

```
"""C 题微网：数据审计、因果预测、情景日前 LP 和 MPC。单位 kWh / 元。"""
from pathlib import Path
import json, hashlib, shutil, datetime as dt
from functools import lru_cache
import numpy as np
import pandas as pd
from openpyxl import load_workbook
from scipy.optimize import linprog
from scipy.sparse import lil_matrix
from scipy.interpolate import PchipInterpolator

ROOT=Path(__file__).resolve().parents[1]
DT=1/6; T=144; SEED=20260910
DATES=pd.date_range('2025-01-01',periods=365)
METHODS=['mean','ma7','ma14','ma30','ewma','ridge']
def dump(obj,path):
    Path(path).write_text(json.dumps(obj,ensure_ascii=False,indent=2,default=lambda x:x.item() if
    isinstance(x,np.generic) else str(x)),encoding='utf-8')
def time_label(t):
    return f'{t//6:02d}:{t%6*10:02d}'
def interval(t,end=None):return time_label(t)+'-'+time_label(t+1 if end is None else end)
def align_time(label):
    # 原模板最后列统一勘误为本日 00:00--00:10: 其他列按左端精确匹配。
    left=str(label).split('-')[0].replace('+1','')
    h,m=map(int,left.split(':')[2:]); return (h*60+m)//10
def _minute(v):
    if isinstance(v,dt.time):return v.hour*60+v.minute
    s=str(v);return 1440 if '+1' in s else int(s.split(':')[0])*60+int(s.split(':')[1])
def load_data():
    if not (ROOT/'input'/'附件 1.xlsx').exists():
        raise FileNotFoundError('请将附件 1 至 4 及附件 5 模板放入本复现目录的 input 文件夹')
    w1=load_workbook(ROOT/'input'/'附件 1.xlsx',data_only=True)
    rows=list(w1.active.values); a=np.asarray([r[1:4] for r in rows[1:]],float)
    assert [_minute(r[0]) for r in rows[1:]]==list(range(10,1441,10))
    def matrix(file,sheet=None):
        w=load_workbook(ROOT/'input'/'附件'+file,data_only=True);s=w[sheet] if sheet else w.active
        r=list(s.values)
        assert [_minute(x) for x in r[0][1:]]==list(range(10,1441,10))
        assert list(pd.to_datetime([x[0] for x in r[1:]]))==list(DATES)
        return np.asarray([x[1:] for x in r[1:]],float)
    L=matrix('附件 2.xlsx','小区负载');PV=matrix('附件 2.xlsx','光伏发电实际功率');P=matrix('附件 4.xlsx')
    w3=load_workbook(ROOT/'input'/'附件 3.xlsx',data_only=True)
    F=np.full((365,4,24),np.nan);mapping=[];seen=set();last=None
    for row in list(w3.active.values)[1:]:
        if row[0] not in [None,'']:last=pd.Timestamp(row[0])
        hour=int(str(row[1]).split(':')[0]); day=(last-DATES[0]).days;assert hour in [0,6,12,18]
        assert (day,hour) not in seen;seen.add((day,hour));F[day,hour//6]=row[2:26]
        for k in
range(24):mapping.append([str(last+pd.Timedelta(hours=hour)),k+1,str(last+pd.Timedelta(hours=hour+k+1))])
    assert len(seen)==1460
    pd.DataFrame(mapping,columns=['发布时间','预报第几小时','目标时间']).to_csv(ROOT/'audit'/'预报时间映
射.csv',index=False,encoding='utf-8-sig')
    stats=[]
    for name,v,unit in [(('附件 1 电价',a[:,0]),'元/kWh'),(('附件 1 负载',a[:,1]),'kW'),(('附件 1 光伏',a[:,2]),'kW'),(('附件 2 负
载',L,'kW'),('附件 2 光伏',PV,'kW'),('附件 3 预报',F,'kW'),('附件 4 电价',P,'元/kWh'))]:
        stats.append(dict(数据=name,形状=str(v.shape),单位=unit,数量=v.size,缺失=int(np.isnan(v).sum()),非有限
=int((~np.isfinite(v)).sum()),负值=int((v<0).sum()),最小=float(v.min()),最大=float(v.max()),均值=float(v.mean())))
    assert np.isfinite(v).all() and (v>=0).all(),name
    pd.DataFrame(stats).to_csv(ROOT/'audit'/'数据审计.csv',index=False,encoding='utf-8-sig')
    manifest={f.name:hashlib.sha256(f.read_bytes()).hexdigest() for f in (ROOT/'input').iterdir()}
    dump(manifest,ROOT/'audit'/'输入 SHA256.json')
    tm=[];templates={}
    for f in (ROOT/'input').glob('result*.xlsx'):
        w=load_workbook(f);s=w['计划购电量'];labels=[s.cell(i,1).value for i in range(2,146)] if f.stem=='result1'
    else [s.cell(1,j).value for j in range(2,146)]
    ids=[align_time(x) for x in labels];assert sorted(ids)==list(range(144))
    for i,(label,t) in enumerate(zip(labels,ids)):tm.append([f.name,i+1,label,interval(t),time_label(t+1),t,'原末
列勘误' if t==0 else '按左端匹配'])
    templates[f.name]={s.title:[s.max_row,s.max_column] for s in w}
    pd.DataFrame(tm,columns=['文件','数据序号','原模板标签','规范区间','附件右端采样时刻','内部索引','说明
']).to_csv(ROOT/'audit'/'模板逐列时间映射.csv',index=False,encoding='utf-8-sig')
    dump(templates,ROOT/'audit'/'模板结构.json')
    np.savez_compressed(ROOT/'results'/'data.npz',a=a,L=L,PV=P,P=P,F=F)
    return dict(a=a,L=L,PV=P,P=P,F=F)

def forecast(y,d,method,fallback):
    """只能访问 y[:d]: 不同候选均遵循相同截止点。"""
```

```

if d==0: return fallback.copy()
h=y[:d]
if method=='mean': v=h.mean(axis=0)
elif method.startswith('ma'): v=h[-int(method[2:]):].mean(axis=0)
elif method=='ewma':
    weight=.8*np.arange(min(d,30)-1,-1,-1);v=np.average(h[-30:],axis=0,weights=weight)
else:
    if d<12: return forecast(y,d,'ma7',fallback)
    def feat(j):
        return np.stack([np.ones(T),y[j-1]/5000,y[j-7]/5000,y[max(0,j-7):j].mean(axis=0)/5000,
            np.full(T,np.sin(2*np.pi*j/7)),np.full(T,np.cos(2*np.pi*j/7))],axis=1)
        idx=range(max(7,d-60),d);X=np.concatenate([feat(j) for j in idx]);z=y[list(idx)].reshape(-1)/5000
        reg=np.eye(X.shape[1])*1;reg[0,0]=1e-6
        coef=np.linalg.solve(X.T@X+reg,X.T@z);v=feat(d)@coef*5000
        # 用历史范围裁剪外推, 禁止用当日实际值设定上下界。
        v=np.clip(v,0,np.maximum(h[-30:].max(axis=0)*1.3,1.))
    return np.maximum(v,0)
def build_forecasts(data):
    out={}
    for m in METHODS:
        out[m]={}
        for k,col in [('L',1),('PV',2)]:
            out[m][k]=np.stack([forecast(data[k],d,m,data['a'][:,col]) for d in range(365)])
    return out
def metrics(pred,actual):
    e=pred-actual;mask=actual>max(1,float(actual.max()))*.01
    return
dict(MAE=float(np.abs(e).mean()),RMSE=float(np.sqrt((e*e).mean())),nMAE=float(np.abs(e).mean()/max(actual.mean(),1)),
MAPE=float(np.mean(np.abs(e[mask]/actual[mask]))*100))

@lru_cache(maxsize=128)
def lp_structure(n,k,eta):
    # G[n], 各情景(C,D,R,W,S_next)[5n], terminal_abs[k], optional plus/minus[2n]
    nvar=n+k*5*n+k*2*n;A=lil_matrix((2*n*k+n,nvar));b=np.zeros(2*n*k+n)
    for s in range(k):
        off=n+s*5*n
        for t in range(n):
            r=2*n*s+t;A[r,t]=1;A[r,off+t]=-1;A[r,off+n+t]=1;A[r,off+2*n+t]=1;A[r,off+3*n+t]=-1
            r=2*n*s+n+t;A[r,off+4*n+t]=1;A[r,off+t]=-eta;A[r,off+n+t]=1/eta
            if t:A[r,off+4*n+t-1]=-1
        plus=n+k*5*n+k
        for t in range(n):A[2*n*k+t,t]=1;A[2*n*k+t,plus+t]=-1;A[2*n*k+t,plus+n+t]=1
    U=lil_matrix((2*k,nvar))
    for s in range(k):
        z=n+k*5*n+s;end=n+s*5*n+5*n-1
        U[2*s,end]=1;U[2*s,z]=-1;U[2*s+1,end]=-1;U[2*s+1,z]=-1
    return A.tocsr(),U.tocsr()

def
optimize_day(net_scenarios,p,s0,eta=.9,capacity=12000,emergency=5,terminal=.5,old=None,fee=.5,refund=True,cyclic=False):
    N=np.atleast_2d(net_scenarios);k,n=N.shape;p=np.asarray(p);A,U=lp_structure(n,k,eta)
    nv=A.shape[1];c=np.zeros(nv);b=np.zeros(A.shape[0]);bu=np.tile([capacity*.5,-capacity*.5],k)
    weights=np.ones(k)/k;c[:n]=p if old is None else 0
    bounds=[(0,None)]*nv;eps=1e-7
    for s in range(k):
        off=n+s*5*n;b[2*n*s:2*n*s+n]=N[s];b[2*n*s+n]=s0
        c[off:off+2*n]=eps/k;c[off+2*n:off+3*n]=emergency*p*weights[s]
        c[n+k*5*n+s]=terminal*p.mean()*weights[s]
        for t in range(n):
            bounds[off+t]=(0,5000*DT);bounds[off+n+t]=(0,5000*DT)
            bounds[off+4*n+t]=(capacity*.1,capacity*.9)
            if cyclic:bounds[off+2*n+t]=(0,0)
            if cyclic:bounds[off+5*n-1]=(s0,s0)
        plus=n+k*5*n+k
    if old is None:
        # 删除交易关系行; 辅助增减量固定零。
        Ause=A[:2*n*k];buse=b[:2*n*k]
        for t in range(2*n):bounds[plus+t]=(0,0)
    else:
        b[2*n*k:]=old;Ause=A;buse=b
        c[plus:plus+n]=(1+fee)*p;c[plus+n:]=(-1+fee if refund else fee)*p

sol=linprog(c,A_ub=U,b_ub=bu,A_eq=Ause,b_eq=buse,bounds=bounds,method='highs',options={'dual_feasibility_tolerance':1e-8,'primal_feasibility_tolerance':1e-8})
if not sol.success:raise RuntimeError(sol.message)
off=n;C=sol.x[off:off+n];D=sol.x[off+n:off+2*n]
assert np.max(np.minimum(C,D))<1e-5,'同时充放电'
return dict(G=np.maximum(sol.x[:n],0),C=C,D=D,S=np.r_[s0,sol.x[off+4*n:off+5*n]],objective=sol.fun)

def recourse(g,net,s,eta=.9,capacity=12000):
    """当前时段测量到达后进行局部能量平衡; 绝不读取下一时段。"""
    surplus=g-net

```

```

if surplus>=0:
    C=min(surplus,5000*DT,max(0,(capacity*.9-s)/eta));D=R=0.;W=surplus-C
else:
    D=min(-surplus,5000*DT,max(0,(s-capacity*.1)*eta));C=W=0.;R=-surplus-D
return C,D,R,W,s+eta*C-D/eta
def interpolate_hourly_forecast(values,anchor,method='linear'):
    x=np.arange(25);y=np.r_[anchor,values];q=np.arange(1,145)/6
    v=np.interp(q,x,y) if method=='linear' else PchipInterpolator(x,y)(q)
    return np.maximum(v,0)
def scenarios(pred,residuals,margin=.5):
    # 最近历史预测残差的真实整日曲线按全天净能量排序选三条代表;不读取目标日。
    if len(residuals)<3:return pred[None,:]
    r=residuals[-30:];order=np.argsort(r.sum(axis=1));ix=np rint(np.array([.15,.5,.85])*(len(r)-1)).astype(int)
    return pred[None,:]+r[order[ix]]*margin*r.std(axis=0)[None,:]
def merge_emergency_intervals(r,tol=1e-8):
    out=[];start=None
    for t in range(len(r)+1):
        active=t<len(r) and r[t]>tol
        if active and start is None:start=t
        if not active and start is not None:out.append((interval(start,t),float(r[start:t].sum())));start=None
    assert abs(sum(x[1] for x in out)-r.sum())<1e-6
    return out

def
run_policy(data,f1,fp,margin=.5,terminal=.5,eta=.9,capacity=12000,emergency=5,updates=(),q3=False,price_actual=False,
price_forecast=False,fee=.5,refund=True,interp='linear',days=None,s0=None,tag='',save=True):
    if days is None:days=range(365)
    days=list(days);s=capacity*.5 if s0 is None else s0;out={k:[] for k in
['G0','G','C','D','R','W','S','plus','minus','PVhat','Lhat','Pdecision']};daily=[];versions=[]
    netactual=(data['L']-data['PV'])*DT
    resid=netactual-(f1-fp)*DT
    for d in days:
        # 正式评价从2月开始。各问题3策略在1月统一按问题2预运行,确保2月初状态可比。
        q3_active=q3 and not (days[0]==0 and d<31)
        p=data['P'][d] if price_actual else data['a'][:,0]
        pp=forecast(data['P'],d,'ma7',data['a'][:,0]) if price_forecast else p
        pv0=fp[d].copy()
        if q3_active:pv0=interpolate_hourly_forecast(data['F'][d,0],data['PV'][d-1,-1] if d else 0,interp)
        pred=(f1[d]-pv0)*DT
        # 问题3用同一发布时刻过去日期的预报残差,避免把日误差套在临近预报。
        if q3_active:
            hist=[]
            for j in range(max(0,d-30),d):
                pj=interpolate_hourly_forecast(data['F'][j,0],data['PV'][j-1,-1] if j else 0,interp)
                hist.append(netactual[j]-(f1[j]-pj)*DT)
            rs=np.asarray(hist).reshape(-1,T)
        else:rs=resid[d]
        m=.5 if d<31 and days[0]==0 else margin;term=.5 if d<31 and days[0]==0 else terminal
        N=scenarios(pred,rs,m)
        plan=optimize_day(N,pp,s,eta,capacity,emergency,term);g0=plan['G'].copy();g=g0.copy()

C=np.zeros(T);D=C.copy();R=C.copy();W=C.copy();plus=C.copy();minus=C.copy();S=np.zeros(T+1);S[0]=s;ph=pv0.copy();lh=f
l[d].copy()
vv=[g0.copy()]
for t in range(T):
    if q3_active and t in [int(h*6) for h in updates]:
        hour=t//6
        pvnew=interpolate_hourly_forecast(data['F'][d,hour//6],data['PV'][d,t-1,interp][:T-t]
rnow=(data['L'][d,max(0,t-6):t]-f1[d,max(0,t-6):t]).mean()
lnew=np.maximum(0,f1[d,t:]+rnow*np.exp(-np.arange(T-t)/36))
prednew=(lnew-pvnew)*DT
rh=[]
        for j in range(max(0,d-30),d):
            hp=interpolate_hourly_forecast(data['F'][j,hour//6],data['PV'][j,t-1,interp][:T-t]
lr=(data['L'][j,max(0,t-6):t]-f1[j,max(0,t-6):t]).mean()
hl=np.maximum(0,f1[j,t:]+lr*np.exp(-np.arange(T-t)/36))
rh.append(netactual[j,t:]-hl-hp)*DT
        N=scenarios(prednew,np.asarray(rh).reshape(-1,T-t),m)
        pl=optimize_day(N,pp[t:],s,eta,capacity,emergency,term,old=g[t:],fee=fee,refund=refund)
        delta=pl['G']-g[t:];plus[t:]=np.maximum(delta,0);minus[t:]=np.maximum(-
delta,0);g[t:]=pl['G'];ph[t:]=pvnew;lh[t:]=lnew
        vv.append(g.copy())
        C[t],D[t],R[t],W[t],s=recurse(g[t],netactual[d,t],s,eta,capacity);S[t+1]=s
        base=float(p@g0);inc=float((1+fee)*p@plus);cancel=float(fee*p@minus);ref=float(p@minus) if refund else
0.;em=float(emergency*p@R)
        total=base+inc+cancel-ref+em

daily.append(dict(day=d,date=STR(DATES[d].date()),G0=g0.sum(),G=g.sum(),R=R.sum(),W=W.sum(),base=base,increase=inc,ca
ncel=cancel,refund=ref,adjustment=inc+cancel-ref,emergency=em,total=total,S0=S[0],Send=S[-
1],C=C.sum(),D=D.sum(),absolute_adjustment=(plus+minus).sum(),surcharge=fee*float(p@(plus+minus))))
    for key,val in
[('G0',g0),('G',g),('C',C),('D',D),('R',R),('W',W),('S',S),('plus',plus),('minus',minus),('PVhat',ph),('Lhat',lh),('P
decision',pp)]:out[key].append(val)
    versions.append(vv)

```

```

        if tag and (d%60==0 or d==days[-1]):print(tag, str(DATES[d].date()), 'cost', round(total, 2), flush=True)
        out={k:np.asarray(v) for k,v in
        out.items()};out['days']=np.array(days);out['eta']=np.array(eta);out['capacity']=np.array(capacity);out['multiplier']=
=np.array(emergency);out['fee']=np.array(fee);out['refund_flag']=np.array(refund)
        df=pd.DataFrame(daily)
        if save and tag:

np.savez_compressed(ROOT/'results'/f'{tag}.npz', **out);df.to_csv(ROOT/'results'/f'{tag}_daily.csv', index=False, encodi
ng='utf-8-sig')
        dump({str(d):[v.tolist() for v in vv] for d,vv in zip(days, versions) if d in
[78,171,265,354]}, ROOT/'results'/f'{tag}_versions.json')
        return out, df

def validate_solution(out, data, df, price_actual=False):
    ids=out['days'];p=data['P'][ids] if price_actual else np.broadcast_to(data['a'][:,0], (len(ids), T))
    G,C,D,R,W,S=[out[k] for k in
['G', 'C', 'D', 'R', 'W', 'S']];eta=float(out['eta']);cap=float(out['capacity']);fee=float(out['fee'])
    bal=G+data['PV'][ids]*DT+D+R-data['L'][ids]*DT-C-W
    expected=(p*out['G0']).sum(axis=1)+(1+fee)*(p*out['plus']).sum(axis=1)+(fee-
int(out['refund_flag'])*(p*out['minus']).sum(axis=1)+float(out['multiplier'])*(p*R).sum(axis=1)
    checks={'能量守恒':np.max(np.abs(bal)), 'SOC 动态':np.max(np.abs(S[:,1:]-S[:, :-1]-eta*C/D/eta)),
    'SOC 边界':max(0, cap*.1-S.min(), S.max()-cap*.9), '充放电上限':max(0, C.max()-5000*DT, D.max()-5000*DT),
    '非负':max(0, -min(v.min() for v in [G,C,D,R,W])), '同时充放电':np.minimum(C,D).max(),
    '跨日连续':np.max(np.abs(S[1:,0]-S[:, :-1,-1])) if np.all(np.diff(ids)==1) and len(ids)>1 else 0.,
    '费用独立核算':np.max(np.abs(expected-df.total.to_numpy())),
    '有效计划交易恒等式':np.max(np.abs(G-out['G0']-out['plus']+out['minus'])),
    '紧急合并守恒':max(abs(sum(v for _,v in merge_emergency_intervals(r))-r.sum()) for r in R),
    '有限数值':float(not all(np.isfinite(x).all() for x in [G,C,D,R,W,S]))}
    result=[dict(检查=k, 误差=float(v), 状态='PASS' if v<1e-6 else 'FAIL') for k,v in checks.items()]
    assert all(x['状态']=='PASS' for x in result), result
    return result

```

```

from model import *
import argparse, time

def main(stage):
    data=load_data()
    if stage in ['all', 'q1']:
        a=data['a'];r=optimize_day((a[:,1]-a[:,2])*DT,a[:,0],6000,cyclic=True,terminal=0)
        G,C,D,S=[r[x] for x in ['G','C','D','S']];W=G+a[:,2]*DT+D-a[:,1]*DT-C
        assert W.min()>-1e-6 and abs(S[-1]-6000)<1e-6
        np.savez_compressed(ROOT/'results'/'q1.npz',G=G,C=C,D=D,S=S,W=W)
        summary=dict(total_energy=G.sum(),cost=a[:,0]@G,initial=S[0],final=S[-1],max_balance_error=float(np.max(np.abs(G+a[:,2]*DT+D-a[:,1]*DT-C-W))),no_battery_cost=float(a[:,0]@np.maximum((a[:,1]-a[:,2])*DT,0)),six_slots={interval(t):G[t] for t in [60,72,84,96,108,120]})
        # 左端样本、日内周期闭合的另一解释, 仅作参考敏感性, 不用于主模板。
        alt=np.roll(a,1,axis=0);rr=optimize_day((alt[:,1]-alt[:,2])*DT,alt[:,0],6000,cyclic=True,terminal=0)
        summary['left_endpoint_alternative_cost']=alt[:,0]@rr['G']
        dump(summary,ROOT/'results'/'q1_summary.json');print('Q1',summary,flush=True)
        pd.DataFrame([dict(区间=interval(t,t+24),充电量=C[t:t+24].sum(),放电量=D[t:t+24].sum()) for t in range(0,144,24)]).to_csv(ROOT/'results'/'表2_问题1.csv',index=False,encoding='utf-8-sig')
    if stage=='q1':return
    if stage in ['all', 'forecast']:
        forecasts=build_forecasts(data)
        np.savez_compressed(ROOT/'results'/'forecasts.npz',**{m+'_'+k:v for m,dic in forecasts.items() for k,v in dic.items()})
        rows=[]
        for m in METHODS:
            for k in ['L','PV']:
                for period,s1 in [(('1月滚动验证',slice(16,31)),('2月至12月外样本',slice(31,None))):
                    rows.append(dict(method=m,variable=k,period=period,**metrics(forecasts[m][k][s1],data[k][s1])))
        pd.DataFrame(rows).to_csv(ROOT/'results'/'预测精度.csv',index=False,encoding='utf-8-sig')
        # 固定相同初始SOC, 连续15日经济回溯; 不重置每一天的SOC。
        tune=[]
        for m in METHODS:
            for margin in [0.,.5,1.]:
                for terminal in [0.,.5,1.]:
_,df=run_policy(data,forecasts[m]['L'],forecasts[m]['PV'],margin=margin,terminal=terminal,days=range(16,31),save=False)

tune.append(dict(method=m,margin=margin,terminal=terminal,cost=df.total.sum(),R=df.R.sum(),Send=df.Send.iloc[-1]))
    print('tuned',m,flush=True)
    td=pd.DataFrame(tune);td['inventory_adjusted_cost']=td.cost-.5*data['a'][:,0].mean()*(td.Send-6000)
    td=td.sort_values('inventory_adjusted_cost');td.to_csv(ROOT/'results'/'1月经济选型.csv',index=False,encoding='utf-8-sig')

    selected=td.iloc[0].to_dict();dump(selected,ROOT/'results'/'selected.json');print('SELECTED',selected,flush=True)
    if stage=='forecast':return

f=np.load(ROOT/'results'/'forecasts.npz');selected=json.loads((ROOT/'results'/'selected.json').read_text(encoding='utf-8'))
m=selected['method'];fl=f[m+'_L'].copy();fp=f[m+'_PV'].copy()
# 所有正式策略1月统一使用事先固定 EWMA: 不会用1月末选型反写1月的SOC。
fl[:31]=f['ewma_L'][:31];fp[:31]=f['ewma_PV'][:31]
common=dict(margin=selected['margin'],terminal=selected['terminal'])
if stage in ['all', 'annual', 'annual_force']:
    validations=[];summaries=[]

experiments=[('q2',False,False,()),('q3_A',True,False,()),('q3_B',True,False,(6,)),('q3_C',True,False,(6,12)),('q3_D',True,False,(6,12,18)),('q42',False,True,()),('q43_A',True,True,()),('q43_B',True,True,(6,)),('q43_C',True,True,(6,12)),('q43_D',True,True,(6,12,18)),('q42_causal_price',False,True,()),('q43_causal_price',True,True,(6,12,18))]
for tag,q3,pa,updates in experiments:
    if stage!='annual_force' and (ROOT/'results'/'f'+tag).npz().exists():
        out=dict(np.load(ROOT/'results'/'f'+tag).npz());df=pd.read_csv(ROOT/'results'/'f'+tag+'_daily.csv')
    else:out,df=run_policy(data,fl,fp,**common,q3=q3,price_actual=pa,price_forecast='causal' in tag,updates=updates,tag=tag)
    validations.extend([dict(model=tag,**v for v in validate_solution(out,data,df,pa))]
    agg=df[df.day>=31].select_dtypes('number').sum().to_dict();agg.pop('day');agg.pop('S0');agg.pop('Send')
    summaries.append(dict(model=tag,**agg,initial_Feb=out['S'][:31,0],final_Dec=out['S'][-1,-1]))
    pd.DataFrame(validations).to_csv(ROOT/'audit'/'物理与费用验证.csv',index=False,encoding='utf-8-sig')
    pd.DataFrame(summaries).to_csv(ROOT/'results'/'年度汇总.csv',index=False,encoding='utf-8-sig')
    if stage in ['all', 'sensitivity']:
        # 24个连续代表日: 四季各6日, 初值取主策略同日已实现SOC; 同一组比较统一初值。
        starts=[76,169,263,352];rows=[]

cases=[('baseline',{}),('eta085',{'eta':.85}),('eta095',{'eta':.95}),('roundtrip090',{'eta':np.sqrt(.9)}),('capacity9600',{'capacity':9600}),('capacity14400',{'capacity':14400}),('capacity18000',{'capacity':18000}),('emergency3',{'emergency':3}),('emergency7',{'emergency':7}),('margin0',{'margin':0}),('margin1',{'margin':1}),('terminal0',{'terminal':0}),('terminal1',{'terminal':1})]

```

```

:0})),('terminal1',{'terminal':1})),('fee025',{'fee':.25})),('fee075',{'fee':.75})),('no_refund',{'refund':False})),('pchip',{'interp':'pchip'})),('error150',{'error150':1}))
for tag,q3 in [('q2',False),('q3_D',True)]:
    base=np.load(ROOT/'results'/f'{tag}.npz')
    for name,kw in cases:
        if not q3 and name in ['fee025','fee075','no_refund','pchip']:continue
        opt=common[kw];cost=em=ener=spill=0.;value=0.
        for start in starts:
            load=f1.copy();pv=fp.copy()
            # 压力试验只改变当时已可用预测的振幅,不读取当日实际误差。
            if name=='error150':load=load*1.10;pv=pv*.90
            cap=opt.get('capacity',12000);sinit=float(np.clip(base['S'][start,0],cap*.1,cap*.9))
            o,d=run_policy(data,load,pv,**opt,q3=q3,updates=(6,12,18) if q3 else
            ()),days=range(start,start+6),s0=sinit,save=False)

validate_solution(o,data,d);cost+=d.total.sum();em+=d.emergency.sum();ener+=d.R.sum();spill+=d.W.sum();value+=.5*data
['a'][:,0].mean()*(d.Send.iloc[-1]-sinit)
rows.append(dict(model=tag,case=name,cost=cost,inventory_adjusted_cost=cost-
value,emergency=em,R=ener,W=spill))
print('sensitivity',tag,name,round(cost,2),flush=True)
pd.DataFrame(rows).to_csv(ROOT/'results'/f'敏感性 24 日.csv',index=False,encoding='utf-8-sig')
print('DONE',stage,flush=True)

if __name__=='__main__':
    p=argparse.ArgumentParser();p.add_argument('--stage',default='all');main(p.parse_args().stage)

```

附录 B review_results.py 源程序

"""独立回读原始 Excel，与复现轨迹比较；不写入任何原始文件。"""
from model import *

```
def main():
    project=ROOT.parents[1]; data=load_data();checks=[];typical=[];events=[];blocks=[]
    q1=np.load(ROOT/'results/q1.npz')
    mapping={'result1_final.xlsx':'q1','result2_final.xlsx':'q2','result3_final.xlsx':'q3_D',
            'result4-2_final.xlsx':'q42','result4-3_final.xlsx':'q43_D'}
    payload={}
    for fname,tag in mapping.items():
        wb=load_workbook(project/fname,data_only=True)
        o=dict(np.load(ROOT/'results'/f'{tag}.npz'));err=0.;counts=0
        if tag=='q1':
            for row in list(wb['计划购电量'].values)[1:]:
                err=max(err,abs(row[1]-o['G'][align_time(row[0])]))
                counts+=1
            for k in range(6):
                row=list(wb['充放电量'].values)[k+1]
                err=max(err,abs(row[1]-o['C'][k*24:(k+1)*24].sum()),abs(row[2]-o['D'][k*24:(k+1)*24].sum()))
            payload[tag]={k:v.tolist() for k,v in o.items()}
        else:
            df=pd.read_csv(ROOT/'results'/f'{tag}_daily.csv')
            for sn,key in [('计划购电量','G0'),('调整购电量','G')]:
                if sn not in wb:continue
                rows=list(wb[sn].values);ids=[align_time(x) for x in rows[0][1:145]]
                assert sorted(ids)==list(range(144)) and len(rows)==335
                for i,row in enumerate(rows[1:],31):
                    assert pd.Timestamp(row[0])==DATES[i]
                    vals=np.asarray(row[1:145],float);assert np.isfinite(vals).all() and vals.min()>=-1e-7
                    money=df.base.iloc[i]+(df.adjustment.iloc[i] if key=='G' else 0)
                    err=max(err,np.max(np.abs(vals-o[key][i,ids])),abs(sum(vals)-row[145]),abs(money-
row[146]));counts+=144
                    s=wb['充放电量']
                    for d in range(31,365):
                        r=2+(d-31)*6
                        assert pd.Timestamp(s.cell(r,1).value)==DATES[d]
                        for k in range(6):
                            err=max(err,abs(s.cell(r+k,3).value-o['C'][d,k*24:(k+1)*24].sum()),abs(s.cell(r+k,4).value-
o['D'][d,k*24:(k+1)*24].sum()))
                            err=max(err,abs(s.cell(r,6).value-o['S'][d,0]),abs(s.cell(r+1,6).value-o['S'][d,-1]))
                        got={};last=None
                        for row in list(wb['紧急购电量'].values)[1:]:
                            if row[0] is not None:last=pd.Timestamp(row[0]);got[last]=[]
                            if row[1] is not None and isinstance(row[2],(int,float)):got[last].append((row[1],row[2]))
                        for d in range(31,365):
                            expected=merge_emergency_intervals(o['R'][d]) or [('无',0)]
                            assert [x[0] for x in got[DATES[d]]]==[x[0] for x in expected]
                            err=max(err,max(abs(a[1]-b[1]) for a,b in zip(got[DATES[d]],expected)))
                        for d in [78,171,265,354]:
                            rec=dict(model=tag,date=str(DATES[d].date()),**{k:float(df[k].iloc[d]) for k in
['G0','G','base','adjustment','emergency','total','R']})
                            rec['initial']=float(o['S'][d,0]);rec['final']=float(o['S'][d,-1])
                            rec['slots']=[{'time':interval(t),'plan':float(o['G0'][d,t]),'final':float(o['G'][d,t])} for t in
[60,72,84,96,108,120]]
                            rec['blocks']=[{'time':interval(k*24,(k+1)*24),'charge':float(o['C'][d,k*24:(k+1)*24].sum()),'discharge':float(o['D']
[d,k*24:(k+1)*24].sum())} for k in range(6)]
                            rec['events']=merge_emergency_intervals(o['R'][d]) or [('无',0.)]
                            typical.append(rec)
                        payload[tag]={'daily':df.iloc[31:].to_dict('records')}
            checks.append(dict(file=fname,compared_interval_values=counts,max_abs_difference=float(err),result='一致' if
err<1e-5 else '需核查'))
    dump(checks,ROOT/'audit/原始 Excel 独立复算.json')
    dump(typical,ROOT/'results/指定日期完整结果.json')
    dump(payload,ROOT/'results/工作簿修订数据.json')
    assert all(x['max_abs_difference']<1e-5 for x in checks),checks
    print(json.dumps(checks,ensure_ascii=False,indent=2))

if __name__=='__main__':main()
```

附录 B q1_certificate.py 源程序

```
"""问题1可复现求解。运行 python solve_q1.py, 依赖 numpy scipy openpyxl matplotlib."""
from pathlib import Path
import sys, json, csv, hashlib
HERE = Path(__file__).resolve().parent
if (HERE / 'pydeps').exists():
    sys.path.insert(0, str(HERE / 'pydeps'))
import numpy as np
import scipy
from scipy.optimize import linprog
import openpyxl

ROOT = HERE.parents[2]
OUT = HERE.parent / '计算结果'
OUT.mkdir(exist_ok=True)

def solve(price, load, pv, eta=.9, power=5000., initial=6000., cyclic=True, method='highs-ds'):
    # x = [grid(144), charge(144), discharge(144), spill(144), energy(145)]
    n = len(price); nv = 5*n+1; dt = 1/6
    A = np.zeros((2*n, nv)); b = np.zeros(2*n)
    for t in range(n):
        A[t, [t, n+t, 2*n+t, 3*n+t]] = [1, -1, 1, -1]
        b[t] = (load[t]-pv[t])*dt
        A[n+t, [n+t, 2*n+t, 4*n+t, 4*n+t+1]] = [-eta, 1/eta, -1, 1]
    bounds = (([0, None])*n+[(0, power*dt)]*(2*n)
               +[(0, float(v*dt)) for v in pv]+[(1200, 10800)]*(n+1))
    bounds[4*n]=(initial, initial)
    if cyclic: bounds[-1]=(initial, initial)
    c=np.r_[price, np.zeros(4*n+1)]
    r=linprog(c, A_eq=A, b_eq=b, bounds=bounds, method=method)
    if not r.success: raise RuntimeError(r.message)
    x=r.x; q, ch, dis, spill=np.split(x[:4*n], 4); e=x[4*n:]
    dual=float(b@r.eqlin.marginals)
    for j, (lo, hi) in enumerate(bounds):
        if lo is not None: dual+=lo*r.lower.marginals[j]
        if hi is not None: dual+=hi*r.upper.marginals[j]
    balance=q+pv*dt+dis-load*dt-ch-spill
    dyn=np.diff(e)-eta*ch+dis/eta
    checks={'balance_max_abs_kwh':float(np.max(np.abs(balance))),
            'storage_max_abs_kwh':float(np.max(np.abs(dyn))),
            'initial_kwh':float(e[0]), 'terminal_kwh':float(e[-1]),
            'minimum_storage_kwh':float(e.min()), 'maximum_storage_kwh':float(e.max()),
            'maximum_charge_kw':float(ch.max()/dt), 'maximum_discharge_kw':float(dis.max()/dt),
            'simultaneous_charge_discharge_slots':int(np.sum((ch>1e-7)&(dis>1e-7))),
            'dual_objective_yuan':dual, 'primal_dual_gap_yuan':float(abs(r.fun-dual)),
            'global_energy_residual_kwh':float(q.sum()+pv.sum()*dt-load.sum()*dt-spill.sum()-(1-eta)*ch.sum()-(1/eta-1)*dis.sum()-(e[-1]-e[0]))}
    return {'cost':float(r.fun), 'purchase':float(q.sum()), 'charge':float(ch.sum()),
            'discharge':float(dis.sum()), 'spill':float(spill.sum()), 'q':q.tolist(),
            'c':ch.tolist(), 'd':dis.tolist(), 'w':spill.tolist(), 'e':e.tolist(), 'checks':checks}

def main():
    p=ROOT/'附件'/'附件 1.xlsx'
    ws=openpyxl.load_workbook(p, data_only=True).active
    rows=list(ws.values)[1:]
    assert len(rows)==144
    a=np.array([r[1:] for r in rows], dtype=float)
    price, load, pv=a.T
    assert np.isfinite(a).all() and (a>=0).all()
    def minute(v):
        if hasattr(v, 'hour'): return v.hour*60+v.minute
        return 1440 if '+1' in v else int(v.split(':')[0])*60+int(v.split(':')[1])
    assert [minute(r[0]) for r in rows]==list(range(10, 1441, 10))
    res=solve(price, load, pv)
    baseline=np.maximum(load-pv, 0)/6
    res.update({'load_total':float(load.sum()/6), 'pv_total':float(pv.sum()/6),
               'baseline_cost':float(price@baseline), 'baseline_purchase':float(baseline.sum()),
               'baseline_spill':float(np.maximum(pv-load, 0).sum()/6),
               'price_min':float(price.min()), 'price_max':float(price.max()),
               'source_sha256':hashlib.sha256(p.read_bytes()).hexdigest(),
               'scipy_version':scipy.__version__, 'numpy_version':np.__version__,
               'price':price.tolist(), 'load':load.tolist(), 'pv':pv.tolist()})
    res['savings']=res['baseline_cost']-res['cost']
    res['savings_pct']=100*res['savings']/res['baseline_cost']
    res['blocks']=[{'period':f'{i*4}:00-{(i+1)*4}:00',
                    'charge':sum(res['c'][i*24:(i+1)*24]),
                    'discharge':sum(res['d'][i*24:(i+1)*24])} for i in range(6)]
    res['selected']=[{'period':f'{h}:00-{h+1}:00', 'q':res['q'][h*6]} for h in [10, 12, 14, 16, 18, 20]]
    sensitivity=[]
    for label, eta, power, initial, cyclic in [
```

```

('充放电各 85%', .85, 5000, 6000, True), ('充放电各 90%', .9, 5000, 6000, True),
('往返效率 90%', np.sqrt(.9), 5000, 6000, True), ('最大功率 4000kW', .9, 4000, 6000, True),
('最大功率 6000kW', .9, 6000, 6000, True), ('初始电量 1200kWh', .9, 5000, 1200, True),
('初始电量 10800kWh', .9, 5000, 10800, True), ('不约束日未回归', .9, 5000, 6000, False)]:
    z=solve(price,load,pv,eta,power,initial,cyclic)
    sensitivity.append({'case':label,'cost':z['cost'],'purchase':z['purchase'],'end':z['e'][-1]})
shifted=solve(np.roll(price,1),np.roll(load,1),np.roll(pv,1))
sensitivity.append({'case':'时刻作区间起点并循环补 0 时',
'cost':shifted['cost'],'purchase':shifted['purchase'],'end':shifted['e'][-1]})
res['sensitivity']=sensitivity
alt=solve(price,load,pv,method='highs-ipm')
res['checks']['independent_algorithm_cost_gap_yuan']=abs(alt['cost']-res['cost'])
assert res['checks']['balance_max_abs_kwh']<1e-6
assert res['checks']['storage_max_abs_kwh']<1e-6
assert res['checks']['primal_dual_gap_yuan']<1e-5
assert res['checks']['simultaneous_charge_discharge_slots']==0
assert abs(res['e'][-1]-6000)<1e-6
(OUT/'summary.json').write_text(json.dumps(res,ensure_ascii=False,indent=2),encoding='utf-8')
def tm(m): return f'{m//60:02d}:{m%60:02d}'
with (OUT/'逐时段策略.csv').open('w',encoding='utf-8-sig',newline='') as f:
    w=csv.writer(f);w.writerow(['序号','时间段','电价元每 kWh','负载 kW','光伏 kW','购电 kWh','充电 kWh','放电 kWh','弃光 kWh','段初 SOC_kWh','段末 SOC_kWh','购电费元'])
    for t in range(144):w.writerow([t+1,tm(10*t)+'-'+tm(10*(t+1)),price[t],load[t],pv[t],res['q'][t],res['c'][t],res['d'][t],res['w'][t],res['e'][t],res['e'][t+1],price[t]*res['q'][t]])
    import matplotlib
    matplotlib.use('Agg')
    import matplotlib.pyplot as plt
    plt.rcParams.update({'font.sans-serif':['Microsoft YaHei'],'axes.unicode_minus':False,'font.size':10})
    t=np.arange(144)/6
    fig,axs=plt.subplots(3,1,figsize=(9,7),sharex=True,constrained_layout=True)
    axs[0].step(t,load,where='post',label='小区负载',color='#283747')
    axs[0].step(t,pv,where='post',label='光伏预测功率',color='#da8b00')
    axs[0].step(t,np.array(res['q'])*6,where='post',label='计划购电功率',color='#2874a6',alpha=.85)
    axs[0].set_ylabel('功率 / kW');axs[0].legend(ncol=3,fontsize=9)
    axs[1].bar(t,np.array(res['c'])*6,width=1/6,color='#2980b9',label='充电')
    axs[1].bar(t,-np.array(res['d'])*6,width=1/6,color='#cb4335',label='放电')
    axs[1].set_ylabel('储能功率 / kW');axs[1].legend(ncol=2)
    axs[2].plot(np.arange(145)/6,res['e'],color='#1e8449',label='储电量')
    axs[2].axhline(1200,color='gray',ls='--');axs[2].axhline(10800,color='gray',ls='--')
    axs[2].set_ylabel('储电量 / kWh');axs[2].set_xlabel('当日时刻 / h')
    ax=axs[2].twinx();ax.step(t,price,where='post',color='#7d3c98',alpha=.55);ax.set_ylabel('电价 / 元每 kWh')
    for a in axs:a.grid(alpha=.2);a.set_xlim(0,24);a.set_xticks(range(0,25,2))
    fig.savefig(OUT/'调度曲线.png',dpi=200);plt.close(fig)
    print(json.dumps({k:res[k] for k in ['cost','purchase','charge','discharge','spill','baseline_cost','savings_pct','blocks','selected','checks','sensitivity']},ensure_ascii=False,indent=2))

if __name__=='__main__':
    sys.stdout.reconfigure(encoding='utf-8')
    main()

```